PropMatch

Property Listing and Management System

Project Report | Database Management Systems

MySQL | 13 Tables | 2 Views | Constraints | SQL Operations

Database: propmatch | Relational design with integrity constraints

Contents

01 Introduction
02 Problem Statement
03 Objectives
04 Technology Stack
05 Database Design
06 Entity-Relationship Design
07 Keys and Constraints
08 SQL Operations
09 Database Views
10 Data Integrity and Verification
11 Advantages
12 Conclusion
13 Future Enhancements

13	2	13	32	8
Tables	Views	User records	Property records	Locations

1 Introduction

PropMatch is a property listing and management system designed to connect property buyers and renters with agents. The system organizes property information, location details, property types, amenities, images, inquiries and site visits in a structured database.

The project uses a relational database to store and manage property-related information. It aims to make property information easier to organize, retrieve and manage.

2 Problem Statement

Managing property listings using separate records can lead to duplicate information, inconsistent data and difficulty retrieving relevant property details. Buyers and renters also need a convenient way to explore available properties and contact agents.

> **How PropMatch addresses this**
> Maintains structured property records, connects related information through database relationships, and supports property searches and management operations.

3 Objectives

01 Store property listings and their details in a structured database.

02 Maintain user, buyer and agent information.

03 Organize properties by location and property type.

04 Store property images and amenities.

05 Maintain buyer inquiries and site-visit records.

06 Track property price changes.

07 Support efficient retrieval, filtering and sorting of property information.

08 Apply database constraints to improve data consistency and integrity.

4 Technology Stack

PropMatch pairs a relational core with a document store, and exposes both through a single full-stack web application written in JavaScript on Node.js.

MySQL 8 (relational core)

Stores structured data across **13 tables and 2 views**: users, agent profiles, properties, locations, inquiries, site visits, price history and login audit. Accessed with the **mysql2** driver using raw SQL queries (no ORM).

Frontend: Next.js (App Router) with React

React-based web interface where buyers, renters and agents browse, filter and manage property listings. Pages and components are built with the **Next.js App Router** inside the same project as the API layer.

Backend: Next.js API routes on Node.js

Server-side route handlers under **app/api** receive application requests and talk to both databases. Authentication uses a **JWT stored in an httpOnly cookie**; database credentials are kept in environment variables.

MongoDB (NoSQL document store)

Runs alongside MySQL as the second database paradigm. It holds the flexible, document-shaped data behind the **property recommendation component**, which does not fit neatly into fixed relational tables.

5 Database Design

The project database is named `propmatch`. It contains **13 tables** and **2 views**.

#	Table	Purpose
1	`users`	Stores user information, email, password hash, role and account status.
2	`agent_profiles`	Stores agent-specific details, agency information and verification status.
3	`properties`	Stores property descriptions, prices, area, listing type, furnishing and availability status.
4	`locations`	Stores city, locality, state, pincode and optional geographic coordinates.
5	`property_types`	Stores property categories such as Apartment, Commercial, Independent House, Plot and Villa.
6	`amenities`	Stores available amenity names.
7	`property_images`	Stores image references associated with properties.
8	`property_amenities`	Connects properties with their amenities.
9	`favorites`	Records properties saved by users.
10	`inquiries`	Stores messages and inquiry statuses associated with properties and buyers.
11	`site_visits`	Stores requested or scheduled property visits.
12	`price_history`	Records previous and updated property prices.
13	`login_audit`	Records login-related audit information.

6 Entity-Relationship Design

The ER diagram represents the entities, their attributes and the relationships between them. Primary keys identify records uniquely, while foreign keys connect related records.

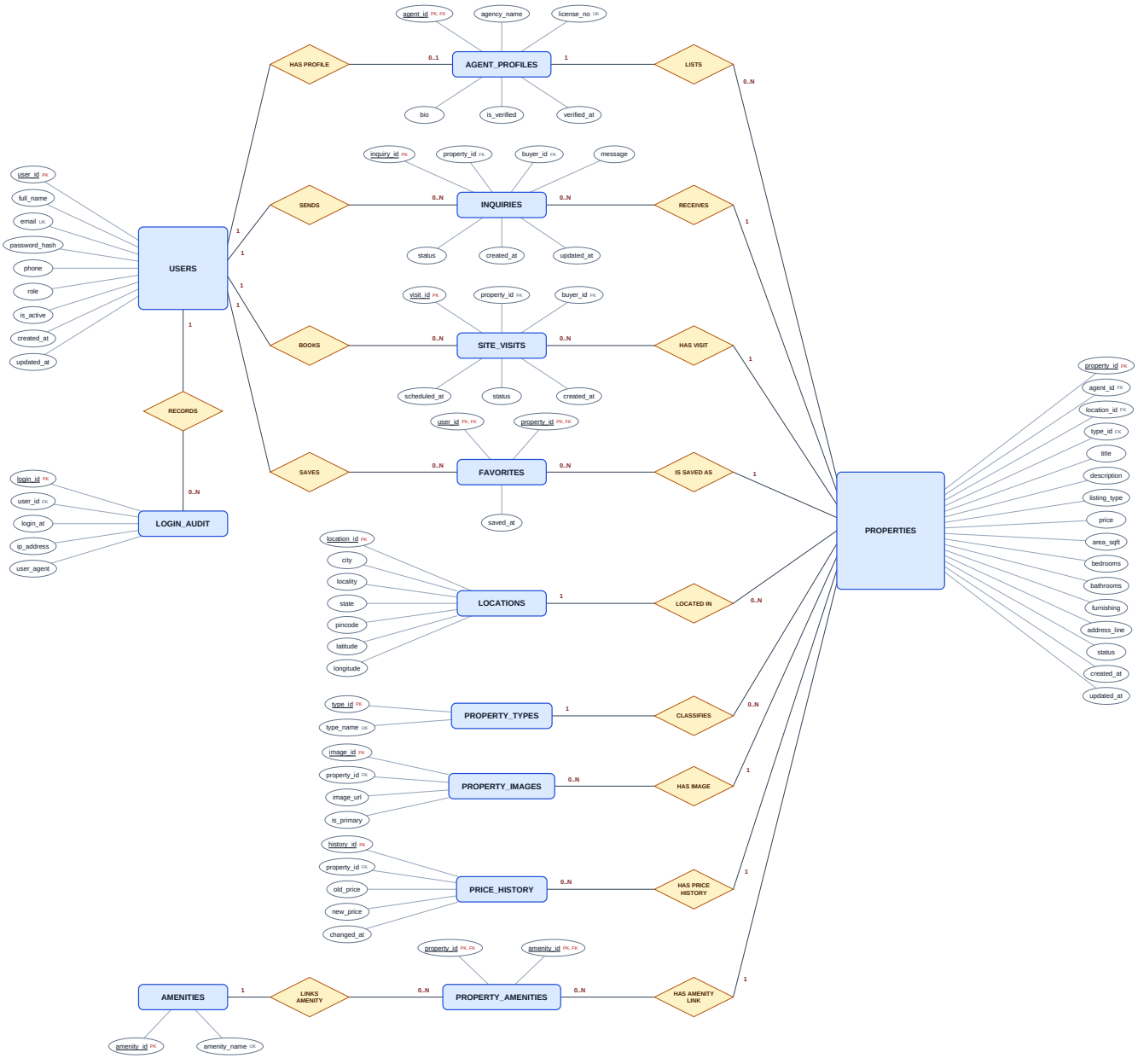


Figure 1. Chen-notation ER diagram of the **`propmatch`** database: 13 entities, 15 relationships, with attributes, keys (PK, FK, UK) and cardinalities (1, 0..1, 0..N).

Relationship Summary

Relationship	Description
User -> Agent profile	A user can have an agent profile.
Agent -> Properties	An agent can list multiple properties.
Location -> Properties	A location can be associated with multiple properties.
Property type -> Properties	A property type can classify multiple properties.
Property -> Images	A property can have multiple images.
User <-> Property (favorites)	Users can save properties through the favorites table.
Buyer -> Inquiries, Visits	Buyers can submit inquiries and request site visits.
Property <-> Amenities	Properties have multiple amenities via property_amenities.
Property -> Price history	Price changes can be recorded in price history.
User -> Login audit	User login events can be recorded in login audit.

7 Keys and Constraints

Constraint	What it does	Example
PRIMARY KEY	Uniquely identifies each record in a table.	property_id identifies a property.
FOREIGN KEY	Maintains relationships between tables.	properties.location_id references locations.location_id.
UNIQUE	Prevents duplicate values in designated columns or combinations.	User email addresses must be unique.
NOT NULL	Ensures required fields cannot be left empty.	Mandatory fields such as price and listing type.
CHECK	Restricts values to specified conditions.	Positive prices and areas; valid coordinate ranges.
DEFAULT	Supplies a predefined value when none is given.	A new property defaults to ACTIVE status.
ENUM	Restricts a column to predefined values.	Listing type is either SALE or RENT.

Why it matters

These constraints help maintain data integrity and consistency. The constraint metadata captured in Section 10 lists every key and constraint declared in the schema.

8 SQL Operations Implemented and Demonstrated

CREATE Create database objects	**INSERT** Add records	**SELECT** Retrieve records
UPDATE Modify records	**DELETE** Remove records	**WHERE** Filter by a condition
ORDER BY Sort retrieved records	**DISTINCT** Retrieve unique values	**GROUP BY** Group for aggregates
COUNT() Count records	**HAVING** Filter grouped results	**JOIN** Combine related tables

Examples demonstrated: filtering properties by price, sorting properties by price, listing distinct property statuses, counting properties by status, and joining property information with location information.

Query Outputs

Screenshots from the MySQL command-line client running against the `propmatch` database.

```
mysql> SELECT * FROM properties LIMIT 5;
+-------------+----------+-------------+---------+-----------------------------+-------------------------------------------------------------------------------------------------------------+--------------+-------------+----
-------+-----------+-----------+------------+-----------------+--------+---------------------+---------------------+
| property_id | agent_id | location_id | type_id | title                       | description                                                                                                 | listing_type | price       | are
a_sqft | bedrooms | bathrooms | furnishing     | address_line | status | created_at          | updated_at          |
+-------------+----------+-------------+---------+-----------------------------+-------------------------------------------------------------------------------------------------------------+--------------+-------------+----
-------+-----------+-----------+------------+-----------------+--------+---------------------+---------------------+
|           1 |        2 |           1 |       1 | Modern Apartment in Whitefield | Spacious 2 BHK apartment with excellent connectivity, parking, lift and security.   | SALE         |  8200000.00 |
  1200 |        2 |         2 | FURNISHED      | NULL         | SOLD   | 2026-09-30 12:55:46 | 2026-09-30 19:19:38 |
|           2 |        2 |           1 |       1 | Premium 3 BHK Whitefield Home  | Premium furnished apartment near IT parks with gym, parking and power backup.       | SALE         | 11500000.00 |
  1650 |        3 |         3 | SEMI_FURNISHED | NULL         | ACTIVE | 2026-09-30 12:55:46 | 2026-09-30 13:00:42 |
|           3 |        2 |           1 |       5 | Luxury Villa Whitefield        | Independent luxury villa with garden, clubhouse and swimming pool facilities.       | SALE         | 25000000.00 |
  2800 |        4 |         4 | FURNISHED      | NULL         | ACTIVE | 2026-09-30 12:55:46 | 2026-09-30 12:55:46 |
|           4 |        2 |           2 |       1 | Affordable HSR Apartment       | Well maintained 2 BHK apartment close to restaurants, schools and public transport. | SALE         |  7400000.00 |
  1100 |        2 |         2 | SEMI_FURNISHED | NULL         | ACTIVE | 2026-09-30 12:55:46 | 2026-09-30 13:00:42 |
|           5 |        2 |           2 |       3 | HSR Independent House          | Spacious independent house with garden, parking and reliable water supply.          | SALE         | 13500000.00 |
  2100 |        3 |         3 | FURNISHED      | NULL         | ACTIVE | 2026-09-30 12:55:46 | 2026-09-30 13:00:42 |
+-------------+----------+-------------+---------+-----------------------------+-------------------------------------------------------------------------------------------------------------+--------------+-------------+----
-------+-----------+-----------+------------+-----------------+--------+---------------------+---------------------+
5 rows in set (0.002 sec)
```

SELECT `SELECT * FROM properties LIMIT 5;` retrieves the first five property records with listing type, price, area, furnishing and status values.

```
mysql> SELECT title, price
    -> FROM properties
    -> WHERE price < 5000000;
+--------------------------+----------+
| title                    | price    |
+--------------------------+----------+
| HSR Rental Apartment     | 42000.00 |
| Koramangala Rental Home  | 65000.00 |
| Koramangala Office Space | 95000.00 |
| Marathahalli Rental Flat | 38000.00 |
| JP Nagar Rental Apartment | 45000.00 |
+--------------------------+----------+
5 rows in set (0.002 sec)
```

WHERE Filters properties priced below 5,000,000. Returns the 5 rental listings.

```
mysql> SELECT title, price
    -> FROM properties
    -> ORDER BY price ASC;
+-----------------------------------+-------------+
| title                             | price       |
+-----------------------------------+-------------+
| Marathahalli Rental Flat          |    38000.00 |
| HSR Rental Apartment              |    42000.00 |
| JP Nagar Rental Apartment         |    45000.00 |
| Koramangala Rental Home           |    65000.00 |
| Koramangala Office Space          |    95000.00 |
| Electronic City 2 BHK             |  6200000.00 |
| Marathahalli Family Apartment     |  6800000.00 |
| Yelahanka New Apartment           |  7200000.00 |
| Affordable HSR Apartment          |  7400000.00 |
| JP Nagar 2 BHK                    |  7500000.00 |
| Modern Apartment in Whitefield    |  8200000.00 |
| Electronic City 3 BHK             |  8500000.00 |
| Marathahalli 3 BHK                |  9200000.00 |
| API Test Property                 |  9500000.00 |
| Agent One Updated Apartment       |  9500000.00 |
| Indiranagar 2 BHK                 |  9800000.00 |
| Electronic City Commercial Space  | 11000000.00 |
| Premium 3 BHK Whitefield Home     | 11500000.00 |
| JP Nagar Premium Apartment        | 11800000.00 |
| Yelahanka Independent House       | 12000000.00 |
| Electronic City Villa             | 13500000.00 |
| HSR Independent House             | 13500000.00 |
| Indiranagar Premium Flat          | 14500000.00 |
| Koramangala Luxury Apartment      | 15500000.00 |
| JP Nagar Independent House        | 15500000.00 |
| Marathahalli Villa                | 16000000.00 |
| Indiranagar Commercial Property   | 18000000.00 |
| Yelahanka Luxury Villa            | 18500000.00 |
| JP Nagar Commercial Building      | 19500000.00 |
| Indiranagar Villa                 | 22500000.00 |
| Luxury Villa Whitefield           | 25000000.00 |
| Koramangala Independent Villa     | 28000000.00 |
+-----------------------------------+-------------+
32 rows in set (0.000 sec)
```

ORDER BY Sorts all 32 properties by price in ascending order, from 38,000 rentals up to 28,000,000 villas.

```
mysql> SELECT DISTINCT status
    -> FROM properties;
+-----------+
| status    |
+-----------+
| ACTIVE    |
| SOLD      |
| RENTED    |
| INACTIVE  |
+-----------+
4 rows in set (0.001 sec)
```

DISTINCT Lists the unique property statuses: ACTIVE, SOLD, RENTED and INACTIVE.

```
mysql> SELECT status, COUNT(*) AS total_properties
    -> FROM properties
    -> GROUP BY status;
+-----------+------------------+
| status    | total_properties |
+-----------+------------------+
| ACTIVE    |               27 |
| SOLD      |                2 |
| RENTED    |                1 |
| INACTIVE  |                2 |
+-----------+------------------+
4 rows in set (0.000 sec)
```

GROUP BY + COUNT() Counts properties per status: 27 ACTIVE, 2 SOLD, 1 RENTED, 2 INACTIVE.

```
mysql> SELECT status, COUNT(*) AS total_properties
    -> FROM properties
    -> GROUP BY status
    -> HAVING COUNT(*) > 1;
+-----------+------------------+
| status    | total_properties |
+-----------+------------------+
| ACTIVE    |               27 |
| SOLD      |                2 |
| INACTIVE  |                2 |
+-----------+------------------+
3 rows in set (0.000 sec)
```

HAVING Keeps only statuses with more than one property. RENTED (1) is filtered out.

```
mysql> SELECT p.title, p.price, l.city, l.locality
    -> FROM properties p
    -> JOIN locations l
    -> ON p.location_id = l.location_id;
+----------------------------------+-------------+-----------+-----------------+
| title                            | price       | city      | locality        |
+----------------------------------+-------------+-----------+-----------------+
| Electronic City 2 BHK            |  6200000.00 | Bengaluru | Electronic City |
| Electronic City 3 BHK            |  8500000.00 | Bengaluru | Electronic City |
| Electronic City Villa            | 13500000.00 | Bengaluru | Electronic City |
| Electronic City Commercial Space | 11000000.00 | Bengaluru | Electronic City |
| Affordable HSR Apartment         |  7400000.00 | Bengaluru | HSR Layout      |
| HSR Independent House            | 13500000.00 | Bengaluru | HSR Layout      |
| HSR Rental Apartment             |    42000.00 | Bengaluru | HSR Layout      |
| Indiranagar Premium Flat         | 14500000.00 | Bengaluru | Indiranagar     |
| Indiranagar 2 BHK                |  9800000.00 | Bengaluru | Indiranagar     |
| Indiranagar Villa                | 22500000.00 | Bengaluru | Indiranagar     |
| Indiranagar Commercial Property  | 18000000.00 | Bengaluru | Indiranagar     |
| JP Nagar 2 BHK                   |  7500000.00 | Bengaluru | JP Nagar        |
| JP Nagar Premium Apartment       | 11800000.00 | Bengaluru | JP Nagar        |
| JP Nagar Independent House       | 15500000.00 | Bengaluru | JP Nagar        |
| JP Nagar Rental Apartment        |    45000.00 | Bengaluru | JP Nagar        |
| JP Nagar Commercial Building     | 19500000.00 | Bengaluru | JP Nagar        |
| Koramangala Luxury Apartment     | 15500000.00 | Bengaluru | Koramangala     |
| Koramangala Independent Villa    | 28000000.00 | Bengaluru | Koramangala     |
| Koramangala Rental Home          |    65000.00 | Bengaluru | Koramangala     |
| Koramangala Office Space         |    95000.00 | Bengaluru | Koramangala     |
| Marathahalli Family Apartment    |  6800000.00 | Bengaluru | Marathahalli    |
| Marathahalli 3 BHK               |  9200000.00 | Bengaluru | Marathahalli    |
| Marathahalli Villa               | 16000000.00 | Bengaluru | Marathahalli    |
| Marathahalli Rental Flat         |    38000.00 | Bengaluru | Marathahalli    |
| Modern Apartment in Whitefield   |  8200000.00 | Bengaluru | Whitefield      |
| Premium 3 BHK Whitefield Home    | 11500000.00 | Bengaluru | Whitefield      |
| Luxury Villa Whitefield          | 25000000.00 | Bengaluru | Whitefield      |
| Agent One Updated Apartment      |  9500000.00 | Bengaluru | Whitefield      |
| API Test Property                |  9500000.00 | Bengaluru | Whitefield      |
| Yelahanka New Apartment          |  7200000.00 | Bengaluru | Yelahanka       |
| Yelahanka Luxury Villa           | 18500000.00 | Bengaluru | Yelahanka       |
| Yelahanka Independent House      | 12000000.00 | Bengaluru | Yelahanka       |
+----------------------------------+-------------+-----------+-----------------+
32 rows in set (0.001 sec)
```

JOIN Joins properties with locations on location_id. Returns 32 rows with city and locality for each listing.

9 Database Views

v_active_listings

Combines property details with related property types, locations and agent information. Filters for active listings associated with verified agents and active user accounts, and exposes calculated price-per-square-foot information.

v_agent_performance

Summarizes agent information and listing statistics, including total listings, active listings, closed listings, inquiries, visits and average listing price.

Views simplify access to information assembled from multiple tables.

```
mysql> SELECT * FROM v_active_listings LIMIT 10;
+-------------+----------------------------------+--------------+-------------+-----------+----------+-----------+-----------+-
```

10 Data Integrity and Verification

The database was checked using table listings, table descriptions, constraint metadata, record counts and view definitions.

- ✓ Main tables contain records (see record counts below).
- ✓ All 13 tables and 2 views are present in the propmatch schema.
- ✓ Primary and foreign key relationships are declared (13 primary keys, 15 foreign keys).
- ✓ UNIQUE and CHECK constraints are present (5 UNIQUE, 10 CHECK).

13	32	8
User records	Property records	Location records

Additional records exist in tables such as inquiries, property images, favorites and site visits. These checks provide evidence that the database schema and basic SQL operations are functioning. Full end-to-end application testing should be documented separately.

```
mysql> SHOW TABLES;
+---------------------+
| Tables_in_propmatch |
+---------------------+
| agent_profiles      |
| amenities           |
| favorites           |
| inquiries           |
| locations           |
| login_audit         |
| price_history       |
| properties          |
| property_amenities  |
| property_images     |
| property_types      |
| site_visits         |
| users               |
| v_active_listings   |
| v_agent_performance |
+---------------------+
15 rows in set (0.014 sec)
```

SHOW TABLES Lists 15 objects in the `propmatch` database: the 13 tables and the 2 views (`v_active_listings`, `v_agent_performance`).

```
mysql> DESCRIBE users;
+---------------+-----------------------------+------+-----+-------------------+-----------------------------------------------+
| Field         | Type                        | Null | Key | Default           | Extra                                         |
+---------------+-----------------------------+------+-----+-------------------+-----------------------------------------------+
| user_id       | int unsigned                | NO   | PRI | NULL              | auto_increment                                |
| full_name     | varchar(100)                | NO   |     | NULL              |                                               |
| email         | varchar(150)                | NO   | UNI | NULL              |                                               |
| password_hash | varchar(255)                | NO   |     | NULL              |                                               |
| phone         | varchar(15)                 | YES  |     | NULL              |                                               |
| role          | enum('BUYER','AGENT','ADMIN') | NO   |     | BUYER             |                                               |
| is_active     | tinyint(1)                  | NO   |     | 1                 |                                               |
| created_at    | timestamp                   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at    | timestamp                   | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+---------------+-----------------------------+------+-----+-------------------+-----------------------------------------------+
9 rows in set (0.005 sec)

mysql> DESCRIBE properties;
+---------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| Field         | Type                                                | Null | Key | Default           | Extra                                         |
+---------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
| property_id   | int unsigned                                        | NO   | PRI | NULL              | auto_increment                                |
| agent_id      | int unsigned                                        | NO   | MUL | NULL              |                                               |
| location_id   | int unsigned                                        | NO   | MUL | NULL              |                                               |
| type_id       | int unsigned                                        | NO   | MUL | NULL              |                                               |
| title         | varchar(150)                                        | NO   | MUL | NULL              |                                               |
| description   | text                                                | NO   |     | NULL              |                                               |
| listing_type  | enum('SALE','RENT')                                 | NO   |     | NULL              |                                               |
| price         | decimal(14,2)                                       | NO   |     | NULL              |                                               |
| area_sqft     | int unsigned                                        | NO   |     | NULL              |                                               |
| bedrooms      | tinyint unsigned                                    | NO   |     | 0                 |                                               |
| bathrooms     | tinyint unsigned                                    | NO   |     | 0                 |                                               |
| furnishing    | enum('UNFURNISHED','SEMI_FURNISHED','FURNISHED')    | NO   |     | UNFURNISHED       |                                               |
| address_line  | varchar(200)                                        | YES  |     | NULL              |                                               |
| status        | enum('ACTIVE','SOLD','RENTED','INACTIVE')           | NO   | MUL | ACTIVE            |                                               |
| created_at    | timestamp                                           | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED                             |
| updated_at    | timestamp                                           | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+---------------+-----------------------------------------------------+------+-----+-------------------+-----------------------------------------------+
16 rows in set (0.012 sec)

mysql> DESCRIBE locations;
+-------------+-------------+------+-----+---------+----------------+
| Field       | Type        | Null | Key | Default | Extra          |
+-------------+-------------+------+-----+---------+----------------+
| location_id | int unsigned | NO   | PRI | NULL    | auto_increment |
| city        | varchar(80) | NO   | MUL | NULL    |                |
| locality    | varchar(100) | NO   |     | NULL    |                |
| state       | varchar(80) | NO   |     | NULL    |                |
| pincode     | char(6)     | NO   |     | NULL    |                |
| latitude    | decimal(9,6) | YES  |     | NULL    |                |
| longitude   | decimal(9,6) | YES  |     | NULL    |                |
+-------------+-------------+------+-----+---------+----------------+
7 rows in set (0.002 sec)
```

DESCRIBE Column definitions for `users`, `properties` and `locations`: data types, NOT NULL settings, keys, defaults, auto-increment and ENUM columns.

```
mysql> SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
    -> FROM information_schema.TABLE_CONSTRAINTS
    -> WHERE TABLE_SCHEMA = 'propmatch'
    -> ORDER BY TABLE_NAME, CONSTRAINT_TYPE;
+--------------------+---------------------+-----------------+
| TABLE_NAME         | CONSTRAINT_NAME     | CONSTRAINT_TYPE |
+--------------------+---------------------+-----------------+
| agent_profiles     | fk_agent_user       | FOREIGN KEY     |
| agent_profiles     | PRIMARY             | PRIMARY KEY     |
| agent_profiles     | uq_agent_license    | UNIQUE          |
| amenities          | PRIMARY             | PRIMARY KEY     |
| amenities          | uq_amenity_name     | UNIQUE          |
| favorites          | fk_fav_property     | FOREIGN KEY     |
| favorites          | fk_fav_user         | FOREIGN KEY     |
| favorites          | PRIMARY             | PRIMARY KEY     |
| inquiries          | chk_inq_message     | CHECK           |
| inquiries          | fk_inq_buyer        | FOREIGN KEY     |
| inquiries          | fk_inq_property     | FOREIGN KEY     |
| inquiries          | PRIMARY             | PRIMARY KEY     |
| locations          | chk_lat             | CHECK           |
| locations          | chk_lng             | CHECK           |
| locations          | chk_pincode         | CHECK           |
| locations          | PRIMARY             | PRIMARY KEY     |
| locations          | uq_location         | UNIQUE          |
| login_audit        | fk_login_audit_user | FOREIGN KEY     |
| login_audit        | PRIMARY             | PRIMARY KEY     |
| price_history      | fk_ph_property      | FOREIGN KEY     |
| price_history      | PRIMARY             | PRIMARY KEY     |
| properties         | chk_prop_price      | CHECK           |
| properties         | chk_prop_area       | CHECK           |
| properties         | chk_prop_baths      | CHECK           |
| properties         | chk_prop_bedrooms   | CHECK           |
| properties         | fk_prop_agent       | FOREIGN KEY     |
| properties         | fk_prop_location    | FOREIGN KEY     |
| properties         | fk_prop_type        | FOREIGN KEY     |
| properties         | PRIMARY             | PRIMARY KEY     |
| property_amenities | fk_pa_amenity       | FOREIGN KEY     |
| property_amenities | fk_pa_property      | FOREIGN KEY     |
| property_amenities | PRIMARY             | PRIMARY KEY     |
| property_images    | fk_img_property     | FOREIGN KEY     |
| property_images    | PRIMARY             | PRIMARY KEY     |
| property_types     | PRIMARY             | PRIMARY KEY     |
| property_types     | uq_type_name        | UNIQUE          |
| site_visits        | fk_visit_buyer      | FOREIGN KEY     |
| site_visits        | fk_visit_property   | FOREIGN KEY     |
| site_visits        | PRIMARY             | PRIMARY KEY     |
| users              | chk_users_email     | CHECK           |
| users              | chk_users_phone     | CHECK           |
| users              | PRIMARY             | PRIMARY KEY     |
| users              | uq_users_email      | UNIQUE          |
+--------------------+---------------------+-----------------+
43 rows in set (0.003 sec)
```

Constraint metadata Query on `information_schema.TABLE_CONSTRAINTS` returns 43 constraints across the schema: 13 PRIMARY KEY, 15 FOREIGN KEY, 5 UNIQUE and 10 CHECK.

```
mysql> SELECT COUNT(*) AS total_users FROM users;
+-------------+
| total_users |
+-------------+
|          13 |
+-------------+
1 row in set (0.001 sec)

mysql> SELECT COUNT(*) AS total_properties FROM properties;
+------------------+
| total_properties |
+------------------+
|               32 |
+------------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_locations FROM locations;
+-----------------+
| total_locations |
+-----------------+
|               8 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_inquiries FROM inquiries;
+-----------------+
| total_inquiries |
+-----------------+
|              17 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_images FROM property_images;
+--------------+
| total_images |
+--------------+
|           31 |
+--------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_favorites FROM favorites;
+-----------------+
| total_favorites |
+-----------------+
|               8 |
+-----------------+
1 row in set (0.000 sec)

mysql> SELECT COUNT(*) AS total_site_visits FROM site_visits;
+-------------------+
| total_site_visits |
+-------------------+
|                18 |
+-------------------+
1 row in set (0.000 sec)
```

Record counts `COUNT(*)` per table: 13 users, 32 properties, 8 locations, 17 inquiries, 31 property images, 8 favorites and 18 site visits.

11 Advantages

- Centralized storage of property information.
- Relationships between properties, users, agents and locations.
- Reduced duplication through normalized related tables.
- Improved consistency through database constraints.
- Easier retrieval and filtering of property listings.
- Support for inquiry, favorites and site-visit records.
- Access to summarized information through database views.

12 Conclusion

PropMatch demonstrates the use of relational database concepts in a property listing and management application. The database includes structured tables, primary and foreign keys, integrity constraints, SQL data-manipulation operations, joins, aggregate queries and views.

The project provides a foundation for managing property information and related user activities. Its overall completeness depends on the successful integration and testing of the frontend, backend, authentication and any additional database features required by the project specification.

13 Future Enhancements

1	Improve property search and filtering.
2	Add more detailed agent and property analytics.
3	Strengthen role-based access control.
4	Improve automated testing and API validation.
5	Extend reporting and property recommendation features.